

PROJECT REPORT

EDUGENIE

Google Gemini Powered AI Learning Assistant

A FastAPI + HTML/CSS/JavaScript Generative AI Application

Project Title	EduGenie — Google Gemini Powered AI Learning Assistant
Technology Stack	Python, FastAPI, Google Gemini API, Jinja2, HTML, CSS, JavaScript
Application Type	Web-based educational assistant
Development Environment	VS Code / Localhost
Prepared By	[Student Name]
Institution / Department	[College / Department]

Academic Project Report

2026

ABSTRACT

EduGenie is a lightweight web-based AI learning assistant designed to help learners understand academic content through generative artificial intelligence. The supplied implementation combines a FastAPI backend with a responsive HTML/CSS/JavaScript interface and a shared Google Gemini client. The system supports five learning tasks: asking questions, explaining concepts, generating quizzes, summarizing text, and recommending structured learning paths. The application accepts a task, learner level, and text input from the browser, sends a validated request to the corresponding FastAPI endpoint, invokes the appropriate AI module, and displays the generated result in the same interface.

The current source code is modular: `main.py` handles application routing and validation; `ai_client.py` centralizes Gemini communication; `qna.py`, `explanation_module.py`, `quiz_module.py`, `summary_module.py`, and `learning_path.py` implement individual educational functions. The explanation module also contains an optional local LaMini-Flan-T5-783M path with Gemini fallback. The provided screenshots demonstrate the browser interface, user input using the Pythagoras Theorem topic, and the resulting beginner-friendly explanation.

Keywords: Generative AI, Google Gemini, FastAPI, Educational Assistant, Question Answering, Quiz Generation, Summarization, Learning Path, Python.

1. INTRODUCTION

1.1 Background

Digital learning increasingly benefits from interactive systems that can respond to a learner's questions, simplify difficult concepts, and adapt content to different levels of understanding. The reference project describes EduGenie as an AI-powered educational assistant intended for students and self-learners. The supplied implementation realizes this idea as a lightweight local web application backed by Google Gemini.

1.2 Problem Statement

Learners often need quick explanations, revision summaries, practice questions, and guidance about what to study next. Traditional static learning material may not provide immediate, personalized interaction. EduGenie addresses this gap by placing multiple AI-assisted learning functions behind a single, simple browser interface.

1.3 Objectives

- Provide a simple browser-based interface for interacting with an AI learning assistant.
- Answer educational questions using a configurable Google Gemini model.
- Explain concepts in simple language according to the learner's level.
- Generate exactly three multiple-choice questions with four options each.
- Summarize educational passages into concise revision points.
- Create beginner-to-advanced learning paths with practice tasks, milestones, and resource types.
- Keep the application modular so individual AI functions can be maintained or extended independently.

1.4 Scope

The project scope covers the local web interface, FastAPI REST endpoints, Gemini integration, task-specific AI modules, input validation, quiz-result rendering, local execution with Uvicorn, and functional testing

through the browser. The current project does not implement user accounts, persistent learner profiles, a database, voice interaction, or a native mobile application.

2. SYSTEM REQUIREMENTS AND TECHNOLOGIES

2.1 Hardware / Runtime Requirements

- A computer capable of running Python 3.10 or newer.
- Internet connectivity for Google Gemini API calls.
- A modern web browser such as Firefox, Chrome, or Edge.
- Sufficient local storage for Python packages and optional local-model files.

2.2 Software Requirements

Component	Role	Project Evidence
Python	Application programming language	main.py and supporting modules
FastAPI	Backend framework and REST API	main.py
Uvicorn	ASGI server for local execution	README / requirements
Jinja2	HTML template rendering	templates/index.html + main.py
Google Gemini API	Generative AI service	ai_client.py
Pydantic	Request validation	TaskRequest in main.py
HTML/CSS/JavaScript	Browser interface and interaction	templates/index.html, static/style.css
python-dotenv	Loads API configuration from .env	main.py

2.3 Project Folder Structure

```

EduGenie-AI/
├── main.py
├── ai_client.py
├── qna.py
├── explanation_module.py
├── quiz_module.py
├── summary_module.py
├── learning_path.py
├── requirements.txt
├── .env.example
├── .gitignore
├── README.md
├── templates/
│   └── index.html
├── static/
│   └── style.css

```

The supplied ZIP contains 13 project files. The structure separates backend routing, AI-provider access, task-specific logic, and frontend assets. This modular arrangement supports easier testing and future maintenance.

3. SYSTEM ARCHITECTURE AND WORKFLOW

3.1 Architectural Overview

The system follows a browser-to-API-to-AI workflow. The user interacts with the HTML interface; JavaScript sends a POST request to the appropriate FastAPI endpoint; the backend validates the task and input; the selected module prepares a prompt; and `ai_client.py` sends the request to the configured Gemini model. The returned result is then rendered in the browser.

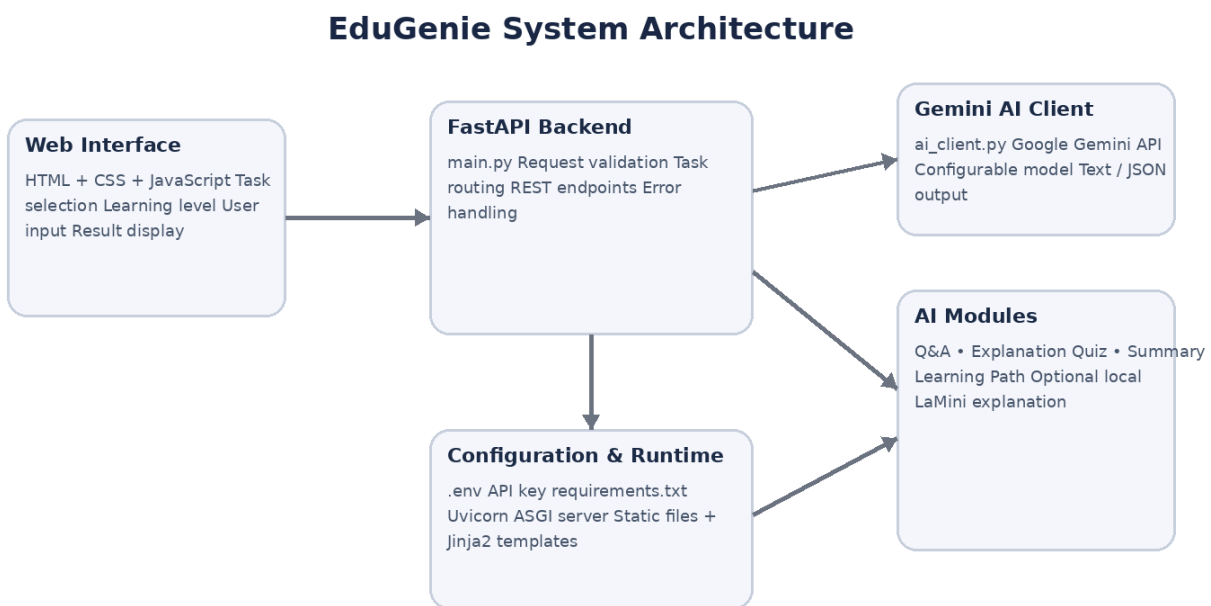


Figure 1. Architectural flow of the supplied EduGenie implementation

3.2 Processing Workflow

1. The learner selects one of five tasks: Explain, Q&A, Quiz, Summarize, or Learning Path.
2. The learner selects Beginner, Intermediate, or Advanced level where applicable and enters the required text.
3. JavaScript selects the matching API endpoint and sends JSON containing task, text, and level.
4. FastAPI validates the request using the TaskRequest Pydantic model.
5. `main.py` routes the request to the corresponding task module.
6. The task module constructs a focused prompt and calls the shared Gemini client.
7. The AI response is returned as text or structured JSON data.
8. The frontend renders the result and, for quizzes, enables interactive answer selection and feedback.

3.3 API Endpoints

Endpoint	Method	Purpose	Output
----------	--------	---------	--------

Endpoint	Method	Purpose	Output
/qa	POST	Answer an educational question	Text
/explain	POST	Explain a concept at a selected level	Text
/quiz	POST	Generate three MCQs	JSON array
/summarize	POST	Create a concise revision summary	Text
/learn/recommendations	POST	Create a structured learning path	Text
/health	GET	Check service status	JSON status

4. MODULE IMPLEMENTATION

4.1 Main Application Module

main.py creates the FastAPI application, loads environment variables from .env, mounts static files, configures Jinja2 templates, defines the TaskRequest schema, and exposes the REST endpoints. The run_task function provides a common routing and error-handling path for all five educational operations.

```
app = FastAPI(
    title="EduGenie",
    version="1.0.0",
    description="Google Gemini-powered learning assistant"
)
```

4.2 Shared Gemini Client

ai_client.py provides a single reusable Gemini client. It reads GEMINI_API_KEY and GEMINI_MODEL from environment variables, caches the client, and requests either text/plain or application/json output. The implementation defaults to the model name gemini-2.5-flash unless GEMINI_MODEL is configured differently.

4.3 Question Answering Module

qna.py instructs the model to behave as a careful educational tutor. The prompt requests accurate, clear answers suitable for the learner's level, with short paragraphs and bullets where useful. If the question is ambiguous, the model is asked to state its assumption rather than inventing facts.

4.4 Explanation Module

explanation_module.py supports two paths. By default it uses Gemini and requests a one-sentence definition, an explanation of how the concept works, an everyday example, and a brief recap. When USE_LOCAL_EXPLANATION=true is configured, the code can use the LaMini-Flan-T5-783M model locally through Transformers. If local inference fails and LOCAL_MODEL_FALLBACK is enabled, the code falls back to Gemini.

4.5 Quiz Generation Module

quiz_module.py asks the AI to create exactly three MCQs, each with four options, a matching correct_answer value, and a short explanation. The response is requested in JSON mode and is validated before being returned. Defensive parsing also attempts to recover a JSON array if a provider wraps it in extra text.

4.6 Summarization Module

summary_module.py requests a concise title and four to seven bullet points for quick revision. The prompt explicitly asks the model to preserve important facts and meaning while avoiding information that is not present in the source passage.

4.7 Learning Path Module

learning_path.py requests a practical learning path organized from beginner to intermediate to advanced. The generated plan includes a suggested timeline, ordered topics, practice tasks, milestones, and resource types such as books, documentation, and videos.

5. FRONTEND DESIGN AND USER INTERFACE

5.1 Browser Interface

The supplied frontend is a responsive single-page interface. It contains a top navigation area, an introductory hero section, a workspace card, task and learner-level selectors, a text area, a Generate button, and a result section. The CSS uses a clean card-based layout with responsive behavior for smaller screens.

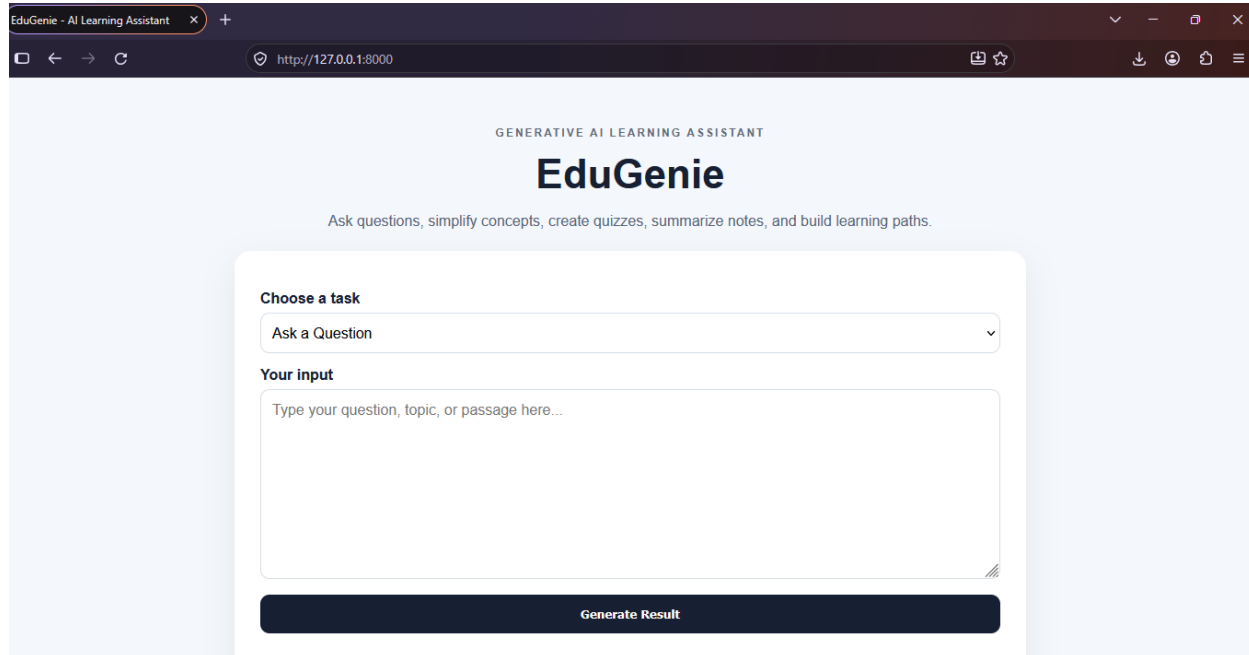


Figure 1. Browser Interface

5.2 Input Interface

For the demonstrated test, the Explain a Concept task is selected at Beginner level and the input is “Pythagoras Theorem.” The interface dynamically changes the input label and placeholder according to the selected task.

The screenshot shows a web browser window with the address bar displaying 'http://127.0.0.1:8000'. The page title is 'EduGenie - AI Learning Assistant'. The main heading is 'EduGenie' with the tagline 'Ask questions, simplify concepts, create quizzes, summarize notes, and build learning paths.' Below this, there is a form titled 'Choose a task' with a dropdown menu set to 'Explain a Concept'. Underneath is a 'Learner level' dropdown menu set to 'beginner'. The 'Your input' section contains a text box with 'Pythagoras Theorem'. At the bottom of the form is a dark blue button labeled 'Generate Result'.

Figure 3. Input — Pythagoras Theorem

5.3 Output Interface

After submission, the result section displays the generated explanation below the input area. The shown response explains the Pythagoras Theorem in simple steps, including the right-angled triangle, hypotenuse, and the formula.

The screenshot shows the same web browser window, but now the 'Generate Result' button is disabled and the text 'Completed.' is displayed above the explanation. The explanation is presented in a light blue box with the following content:

****The Pythagoras Theorem is a mathematical rule that helps you find the length of one missing side of a right-angled triangle if you already know the lengths of the other two sides.****

Here is how it works, step by step:

****Step 1: Meet the right-angled triangle****
A right-angled triangle is a triangle where one of the corners forms a "L" shape (a 90-degree angle).

****Step 2: Know the special side****
In this type of triangle, the longest side is always the one opposite the "L" shaped corner. It has a fancy math name called the ****hypotenuse****, but you can just think of it as the "slanted long side."

****Step 3: The magic formula****
Pythagoras discovered a cool secret about the sides of this triangle. If you take the two shorter sides, multiply each of them by themselves, and add those two numbers together, it will equal the longest side multiplied by itself.
In math speak, we write this as: $a^2 + b^2 = c^2$

Figure 4. Output — Generated Explanation

6. FUNCTIONAL TESTING

6.1 Test Cases

Test ID	Feature	Input	Expected Result	Status
TC01	Browser loading	Open http://127.0.0.1:8000	EduGenie interface loads	Passed
TC02	Concept explanation	Explain + Beginner + Pythagoras Theorem	Simple concept explanation is displayed	Passed
TC03	Q&A	Educational question	AI answer is displayed	Supported
TC04	Quiz	Topic/passage	Three MCQs with four options each	Supported
TC05	Summarization	Educational passage	Concise title and 4–7 bullets	Supported
TC06	Learning path	Topic + learner level	Beginner-to-advanced plan	Supported
TC07	Health endpoint	GET /health	JSON service status	Supported

The first two test cases are directly demonstrated by the supplied screenshots: the browser interface loads successfully and the Explain a Concept workflow produces an output for the Pythagoras Theorem input. The remaining cases are documented capabilities implemented by the supplied source code; they should be exercised with a configured Gemini API key during a full live test.

6.2 Error Handling and Validation

- TaskRequest restricts the task value to explain, qa, quiz, summarize, or learn.
- Input text must contain at least two characters and is limited to 20,000 characters.
- The shared AI client raises a clear error if the Gemini API key is missing.
- The quiz module validates that exactly three questions are returned and that every question has exactly four options.
- Backend AI failures are converted to HTTP 502 responses instead of exposing provider internals or credentials.
- The browser displays a helpful error message when an API request fails.

7. INSTALLATION AND EXECUTION

The supplied README provides the following local execution workflow for Windows/VS Code:

9. Install Python 3.10 or newer and ensure Python is available on PATH.
10. Open the EduGenie-AI folder in VS Code and create a virtual environment.
11. Activate the virtual environment in the VS Code terminal.
12. Install dependencies using `pip install -r requirements.txt`.

13. Create a .env file from .env.example and set GEMINI_API_KEY. The README also supports configuring GEMINI_MODEL.
14. Start the server using uvicorn main:app --reload.
15. Open <http://127.0.0.1:8000> in a browser. The README also lists /docs for Swagger UI and /health for a health check.

```
python -m pip install --upgrade pip
pip install -r requirements.txt
uvicorn main:app --reload
```

Security note: the supplied README explicitly advises keeping the .env file private and not committing the Gemini API key to source control.

8. ADVANTAGES, LIMITATIONS AND FUTURE ENHANCEMENTS

8.1 Advantages

- Single interface for multiple AI-assisted learning tasks.
- Modular Python architecture that separates routing, AI-provider access, and educational functions.
- Level-aware prompts for Beginner, Intermediate, and Advanced learners.
- Structured quiz validation improves consistency of quiz output.
- Responsive frontend with interactive quiz feedback and a copy-result function.
- Lightweight local deployment using FastAPI and Uvicorn.

8.2 Current Limitations

- The application depends on a configured Gemini API key for its default AI workflow.
- AI-generated information can contain errors and should be checked against trusted learning material.
- There is no persistent database or user-account system in the supplied code.
- Learner progress is not stored between sessions.
- The optional local LaMini path requires additional large model dependencies and downloads.

8.3 Future Enhancements

- Voice-based question and answer for hands-free learning.
- Multilingual explanations and summaries.
- Mobile application support.
- Learner accounts, progress tracking, and personalized dashboards.
- Gamification through badges, streaks, and challenge-based assessments.
- Image and PDF input for snapshot-based doubt solving and summarization.
- Teacher/parent dashboards and Learning Management System integration.
- Adaptive learning paths based on learner performance and historical activity.

9. CONCLUSION

EduGenie demonstrates how generative AI can be integrated into a compact educational web application. The supplied implementation combines a FastAPI backend, a responsive HTML/CSS/JavaScript frontend, and

a shared Gemini client to provide five core learning functions: question answering, concept explanation, quiz generation, summarization, and learning-path recommendations.

The modular design makes the system understandable and extensible. The provided screenshots confirm the working browser interface and the complete Explain a Concept interaction for the Pythagoras Theorem example. The code additionally provides structured validation for quiz generation, environment-based AI configuration, API health checking, and optional local explanation-model support.

Overall, EduGenie provides a practical foundation for an intelligent learning companion. Its future development can focus on personalization, accessibility, multilingual interaction, learner analytics, and richer educational content inputs while retaining the current modular architecture.

APPENDIX A — KEY SOURCE FILES

The following table summarizes the role of the major files in the supplied project archive.

File	Responsibility
main.py	FastAPI application, request model, routes, task dispatch, and error handling.
ai_client.py	Shared Google Gemini client and response generation configuration.
qna.py	Educational question-answering prompt and generation.
explanation_module.py	Concept explanation using Gemini or optional local LaMini model.
quiz_module.py	Three-question MCQ generation, JSON parsing, and validation.
summary_module.py	Concise educational summarization.
learning_path.py	Structured beginner-to-advanced learning recommendations.
templates/index.html	Web page structure and browser-side task submission/rendering.
static/style.css	Responsive visual styling for the web interface.
requirements.txt	Python dependency constraints.
README.md	Installation, execution, testing, and project usage instructions.